

Project Executable Files

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	3 Marks

Project Executable Files

Submission of all executable and deployable files for the project, organized for the evaluator to run or deploy directly.

Step 1: Submission Checklist

S.NO	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide	Yes
3	requirements.txt	Yes
4	Database schema / seed files	NA (no database; model files used instead)
5	Environment configuration file (.env.example or similar)	NA (no secrets/API keys required)
6	Deployed application URL	Yes https://risingwaters-1.onrender.com/
7	APK / Executable binary	NA (web app, not mobile/desktop)
8	Dockerfile / Containerization config	No (deployed via Procfile + Gunicorn instead)
9	Test files and test results	Yes (Postman collection)
10	Demo video or walkthrough	Yes (video demo script provided)

Step 2: File / Folder Structure

Rising-Waters/ app.py, requirements.txt, Procfile, floods.save, transform.save, architecture.png, README.md | dataset/flood dataset.xlsx | notebooks/Rising_Waters.ipynb (+ comparison model .pkl files) | templates/home.html, predict.html, chance.html, no_chance.html | static/main.css, main.js, hero-rain.png

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://risingwaters-1.onrender.com/
Login Credentials (Demo)	Not applicable - no authentication required
Platform / Hosting Provider	Render
Repository Link	https://github.com/Laharisrikotipalli/RisingWaters
Demo Video Link	https://drive.google.com/file/d/1mrpNulO4c-xLdWXaaDxH9CV9T5jWIXNK/view?usp=sharing

Step 4: Run Instructions

1. git clone <https://github.com/Laharisrikotipalli/RisingWaters.git>
2. cd Rising-Waters
3. pip install -r requirements.txt
4. python app.py
5. Open <http://127.0.0.1:5000>

Step 5: Known Issues / Limitations

S.NO	Known Issue / Limitation	Workaround / Status
1	No live weather API integration; all values entered manually	Planned as a future enhancement
2	Missing form fields return a generic Flask 400 error	Could be improved with custom form validation messaging
3	No automated test suite (unit/integration tests)	Currently tested manually via Postman/curl